TaskFlow - Session 1 Project Guide

A Simple Guide to the HTML and CSS We Built

This guide explains the official TaskFlow project at the end of Session 1. We will add JavaScript functionality in later sessions.

Keep this guide. You can read it at home to remember what we did.

Everything in this guide comes from the real project files. Nothing is invented.

---

1. What We Built

In Session 1 we built the **look** of TaskFlow.

TaskFlow is a small page that shows tasks.

Here is what the page has:

- A **header** at the top with the title "TaskFlow" and a short line of text.
- Three **statistic cards**: Total Tasks, Completed, and Pending.
- Three **filter buttons**: All, Completed, and Pending.
- A **task list** with six tasks.
- A **badge** on each task that says "Completed" or "Pending".
- **Colors**: green badges for completed tasks, orange badges for pending tasks.
- A **hover** effect on the buttons.
- A **responsive layout**. The page changes on small screens.

What does NOT work yet

This is important. Please read it.

- The filter buttons **do not filter** anything yet. You can click them and nothing happens.
- The numbers 6, 2 and 4 are **typed by hand** in the HTML file. They do not count anything.
- The six tasks are also **typed by hand** in the HTML file.
- The "All" button looks blue, but that is only a color. It is not really selected.
- The file `js/app.js` is empty. It does nothing yet.

This is normal. Session 1 is about structure and design only.

We will make these parts work in Session 2 with JavaScript.

2. Project Files

Our project has three files:

```
project/
├── index.html
├── css/
│   └── style.css
└── js/
    └── app.js
```

index.html This file has the structure and the content of the page. It says what is on the page.

css/style.css This file controls how the page looks. Colors, sizes, and spacing are here.

js/app.js This file is for JavaScript. Right now it only has a comment inside. It does nothing. We will start using it in Session 2.

Keep this structure. Do not move or rename these files.

---

3. HTML and CSS: What Is the Difference?

This is the most important idea of Session 1.

HTML = what is on the page.

CSS = how it looks.

Think about a person:

- HTML is the person.
- CSS is the clothes.

Here is the flow:

```
        HTML
"What is on the page?"
          |
          v
         CSS
 "How should it look?"
          |
```

```
          v
       Browser
    Shows the page
```

Example from our project.

The HTML says "this is a card":

```
<div class="stat-card">
```

The CSS says "cards are white with round corners":

```
.stat-card {
    background-color: #ffffff;
    border-radius: 10px;
}
```

---

4. Basic HTML Page Structure

Here is the top of our real `index.html`:

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>TaskFlow</title>

    <link rel="stylesheet" href="css/style.css">
</head>

<body>
```

Let us look at each line.

`<!DOCTYPE html>` This tells the browser: this is a modern HTML page. It must be the first line.

`<html lang="en">` This starts the page. `lang="en"` says the page is in English.

`<head>` This part holds information **about** the page. You do not see it on the screen.

`<meta charset="UTF-8">` This tells the browser which letters and symbols we use. Without it, some characters can look wrong.

`<meta name="viewport" content="width=device-width, initial-scale=1.0">` This is for phones. It tells the browser to use the real width of the phone screen. Without this line, the page looks very small on a phone.

`<title>TaskFlow</title>` This is the name in the browser tab. It is not shown on the page itself.

`<link rel="stylesheet" href="css/style.css">` This connects our CSS file to the page.

`<body>` This part holds everything people **see**.

---

5. Connecting CSS to HTML

The link

Our CSS file is connected with this line:

```
<link rel="stylesheet" href="css/style.css">
```

`href="css/style.css"` is the **path**. It is the way to the file.

Our files look like this:

```
index.html
   |
   └── css/style.css
```

So the browser reads: "start where index.html is, go into the folder `css`, then take the file `style.css`."

If this path is wrong, the page has **no styles**. It looks plain and white.

Classes

A **class** is a name we give to an element.

In the HTML we write the class:

```
<div class="stat-card">
```

In the CSS we use that name to find it:

```
.stat-card {
    padding: 20px;
}
```

The HTML says: this element is a `stat-card`.

The CSS says: find every `stat-card` and give it this style.

Why the dot?

In CSS, a **dot** `.` before a name means "class".

```
.stat-card     /* an element with class="stat-card" */
stat-card      /* WRONG - this looks for a <stat-card> tag */
```

Remember: **in CSS, class names always start with a dot.**

In HTML there is no dot. You only write the name.

One class, many elements

Many elements can use the same class.

We have three cards, and all three use `class="stat-card"`.

So we write the CSS **one time**, and all three cards get it.

---

6. Global CSS Setup

This is the first rule in our `style.css`:

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

`*` The star means **all elements**. Every single one.

`margin: 0;` Remove the space **outside** elements that the browser adds by itself.

`padding: 0;` Remove the space **inside** elements that the browser adds by itself.

Why do we do this? The browser already puts space around things like headings and paragraphs. That space is different in every browser. We remove it, so we start clean and we decide the spacing ourselves.

`box-sizing: border-box;` This one is harder, so here is an example.

Imagine a box with `width: 100px` and `padding: 20px`.

- Without `border-box`: the box becomes 140px wide. The padding is added on top.
- With `border-box`: the box stays 100px wide. The padding fits inside.

So `border-box` means: **padding and border stay inside the width.**

This makes sizes much easier to understand. We use it in every project.

---

7. Body Styles

```
body {
    font-family: Arial, Helvetica, sans-serif;
    background-color: #f4f5f7;
    color: #1f2430;
    line-height: 1.5;
}
```

`body` selects the whole visible page. These styles work everywhere.

`font-family: Arial, Helvetica, sans-serif;` This is a list of fonts. The browser tries Arial first. If Arial is not available, it tries Helvetica. If that is also not available, it uses another simple sans-serif font.

We write a list because not every computer has the same fonts.

`background-color: #f4f5f7;` This is the page background. It is a very light grey.

`#f4f5f7` is a **hex color**. It is a code for a color. Our cards are white, so the light grey background makes them easy to see.

`color: #1f2430;` This is the **text** color. It is a very dark grey.

We do not use pure black because dark grey is softer for the eyes.

`line-height: 1.5;` This is the space between lines of text. `1.5` means one and a half times the font size. More space makes text easier to read.

---

8. The Main Container

```
.container {
    max-width: 700px;
    margin: 0 auto;
    padding: 30px 20px;
}
```

`.container` holds the main content of the page.

`max-width: 700px;` This stops the content from becoming too wide.

On a big screen, text that goes from the far left to the far right is hard to read. Your eyes have to travel too far.

`max-width` means "never wider than this". But it **can** be smaller. On a phone it just becomes narrower.

This is different from `width: 700px`, which would force 700px and break on a phone.

`margin: 0 auto;` This has two values:

- `0` is the space on the top and bottom.
- `auto` is the space on the left and right.

`auto` means the browser shares the free space equally on both sides. The result is that the box sits in the **center**.

`padding: 30px 20px;` This is space **inside** the container:

- `30px` on the top and bottom.
- `20px` on the left and right.

The left and right padding is important on a phone. Without it, the text would touch the edge of the screen.

---

9. The Header

The HTML

```
<!-- The top bar of the application. -->
<header class="header">
    <h1>TaskFlow</h1>
    <p>All your tasks in one place.</p>
</header>
```

`<header>` This is a real HTML element. It means "the top part of the page".

`<h1>` This is the most important heading on the page. There should be only one `<h1>` on a page.

`<p>` This is a paragraph of text.

`<!-- ... -->` This is a **comment**. The browser ignores it. We write comments to explain the code to people.

The CSS

```
.header {
    background-color: #ffffff;
    border-bottom: 1px solid #e2e5ea;
    padding: 30px 20px;
    text-align: center;
}
```

`background-color: #ffffff;` makes the bar white.

`border-bottom: 1px solid #e2e5ea;` draws a thin line under the header. It has three parts: `1px` is how thick, `solid` is the style, `#e2e5ea` is the color.

`padding: 30px 20px;` gives space inside: 30px top and bottom, 20px left and right.

`text-align: center;` puts the text in the middle.

Now the two rules for the text inside:

```
.header h1 {
    color: #3b6ef5;
    font-size: 32px;
```

```
}

.header p {
    color: #6b7280;
    margin-top: 6px;
}
```

`.header h1` has a **space** between the two names. A space means "inside". So this means: the `<h1>` that is **inside** `.header`.

This is useful. It styles only this heading, not every heading on the page.

- `color: #3b6ef5;` makes the title blue.
- `font-size: 32px;` makes it big.

`.header p` is the paragraph inside the header.

- `color: #6b7280;` makes it grey, so it looks less important than the title.
- `margin-top: 6px;` adds a small space above it.

---

10. Statistic Cards

The HTML

```
<!-- The cards that show our numbers. -->
<section class="stats">

    <div class="stat-card">
        <p class="stat-label">Total Tasks</p>
        <p class="stat-value">6</p>
    </div>

    <div class="stat-card">
        <p class="stat-label">Completed</p>
        <p class="stat-value">2</p>
    </div>

    <div class="stat-card">
        <p class="stat-label">Pending</p>
        <p class="stat-value">4</p>
```

```
    </div>

</section>
```

`<section>` groups things that belong together. Here it groups the three cards.

`<div>` is a simple box. It has no special meaning. We use it for each card.

Each card has two paragraphs:

- `stat-label` is the small grey word, like "Total Tasks".
- `stat-value` is the big number, like 6.

Notice that all three cards use the **same** class `stat-card`. That is why we write the card style only once.

Remember: the numbers 6, 2 and 4 are typed by hand. They do not count anything yet.

The CSS

```
.stats {
    display: flex;
    gap: 16px;
}
```

`display: flex;` This puts the cards next to each other, in one row.

Very important: we put `display: flex` on the **parent** (`.stats`), not on the cards. Flexbox is always set on the box that **contains** the items.

`gap: 16px;` This adds 16px of space **between** the cards. It does not add space outside the group. Only between.

Here is the difference:

```
Without Flexbox:        With Flexbox:

[Card]                  [Card] [Card] [Card]
[Card]
[Card]
```

Normally boxes sit on top of each other. Flexbox puts them side by side.

```
.stat-card {
    flex: 1;
    background-color: #ffffff;
    border: 1px solid #e2e5ea;
    border-radius: 10px;
    padding: 20px;
    text-align: center;
}
```

`flex: 1;` This gives each card an **equal share** of the row. All three cards become the same width, even though the words inside are different lengths.

This is why the cards look tidy.

`background-color: #ffffff;` makes the card white.

`border: 1px solid #e2e5ea;` draws a thin light-grey line around the card.

`border-radius: 10px;` makes the corners round. A bigger number means rounder corners.

`padding: 20px;` adds space inside the card, on all four sides. One value means all four sides are the same.

`text-align: center;` puts the text in the middle of the card.

```
.stat-label {
    color: #6b7280;
    font-size: 14px;
}
```

- `color: #6b7280;` is grey. The label is less important than the number.
- `font-size: 14px;` makes it small.

```
.stat-value {
    font-size: 30px;
    font-weight: bold;
    margin-top: 5px;
}
```

- `font-size: 30px;` makes the number big.
- `font-weight: bold;` makes it thick.
- `margin-top: 5px;` adds a small space above the number.

Big and bold means "this is the important part".

---

11. Filter Buttons

The HTML

```
<!-- The buttons that will choose which tasks we see. -->
<section class="filters">
    <button class="filter-button active">All</button>
    <button class="filter-button">Completed</button>
    <button class="filter-button">Pending</button>
</section>
```

`<button>` is a real HTML button. People can click it.

All three buttons have the class `filter-button`. So all three get the same look.

The first button has **two** classes: `filter-button` **and** `active`.

We write two classes with a space between them:

```
class="filter-button active"
```

The `active` class makes that one button blue.

The CSS

```
.filters {
    display: flex;
    gap: 10px;
    margin-top: 30px;
}
```

- `display: flex;` puts the buttons in one row.
- `gap: 10px;` adds space between them.
- `margin-top: 30px;` adds space above the whole group, so it is not too close to the cards.

```
.filter-button {
    background-color: #ffffff;
    border: 1px solid #e2e5ea;
    border-radius: 8px;
    color: #1f2430;
    cursor: pointer;
    font-size: 15px;
    padding: 10px 18px;
}
```

- `background-color: #ffffff;` makes the button white.
- `border: 1px solid #e2e5ea;` draws a thin grey line around it.
- `border-radius: 8px;` makes the corners a little round.
- `color: #1f2430;` sets the text color to dark grey.
- `cursor: pointer;` changes the mouse pointer to a **hand**. This tells people they can click.
- `font-size: 15px;` sets the text size.
- `padding: 10px 18px;` gives space inside: 10px top and bottom, 18px left and right. This is what makes the button bigger than the word inside it.

```
/* Give the user feedback when the mouse is over a button. */
.filter-button:hover {
    border-color: #3b6ef5;
}
```

`:hover` means these styles are used **while the mouse is over** the button.

When you move the mouse away, the button goes back to normal.

Here the border turns blue. This tells the user "you can click this".

```
/* The button of the filter that is currently selected. */
.filter-button.active {
    background-color: #3b6ef5;
    border-color: #3b6ef5;
    color: #ffffff;
}
```

`.filter-button.active` has **no space** between the two names.

No space means: an element that has **both** classes at the same time.

Our first button has `class="filter-button active"`, so this rule finds it.

- The background becomes blue.
- The border becomes blue.
- The text becomes white.

This is a very common mistake, so look at it carefully:

```
.filter-button.active     /* both classes on ONE element  - correct here */
.filter-button .active    /* .active INSIDE .filter-button - different! */
```

Important

The buttons look like filters, but they do not filter tasks yet.

Clicking them does nothing right now.

We will make them work using JavaScript in Session 2.

---

12. The Task List

The HTML

```
<!-- The list of tasks. -->
<ul class="task-list">

    <li class="task-item">
        <span class="task-title">Design the TaskFlow page</span>
        <span class="task-status completed">Completed</span>
    </li>

    <li class="task-item">
        <span class="task-title">Style the statistic cards</span>
        <span class="task-status pending">Pending</span>
    </li>

</ul>
```

(The real file has six tasks. Two are shown here as an example.)

`<ul>` This is a list. `ul` means "unordered list", which means a list with no numbers.

`<li>` This is **one item** inside the list. `li` means "list item".

Only `<li>` can go directly inside `<ul>`.

Why do we use a list?

Because our tasks really are a list. They are many things of the same kind.

Using `<ul>` and `<li>` tells the browser and other people what this really is. It is better than using many `<div>` boxes.

`<span>` This is a small box for text inside a line. It is like a `<div>`, but it stays in the same line instead of starting a new one.

Each task has two spans:

- `task-title` is the name of the task.
- `task-status` is the small badge that says Completed or Pending.

The CSS

```
.task-list {
    list-style: none;
    margin-top: 20px;
}
```

`list-style: none;` This removes the bullet points (the small dots) that lists have by default.

We remove them because our tasks are cards, not simple text lines.

`margin-top: 20px;` adds space above the list.

The tasks are written by hand

Right now, we write every task manually in `index.html`.

If we want a seventh task, we have to copy an `<li>` block and change the words.

Later, JavaScript will create these task rows for us from data.

One small note

The class `task-title` is written in the HTML, but it has **no CSS rule** in Session 1.

That is not a mistake. The title looks correct because of the styles on `.task-item`. The class is there and ready if we want to style it later.

---

13. Completed and Pending Badges

Look at this line again:

```
<span class="task-status completed">Completed</span>
```

This span has **two** classes: `task-status` and `completed`.

Here is the idea:

```
   task-status
        +
    completed
        =
Completed badge
```

`task-status` = the shared badge styles. Every badge gets these.

`completed` = the extra styles only for completed tasks.

The same idea for pending:

```
  task-status
       +
    pending
       =
Pending badge
```

The CSS

```
.task-status {
    border-radius: 20px;
    color: #ffffff;
    background-color: #6b7280;
    font-size: 12px;
    padding: 5px 12px;
}
```

These are the **shared** styles.

- `border-radius: 20px;` The number is big and the box is small, so the badge becomes a **pill** shape.
- `color: #ffffff;` makes the text white.
- `background-color: #6b7280;` is a grey background.
- `font-size: 12px;` makes the text small.
- `padding: 5px 12px;` gives space inside: 5px top and bottom, 12px left and right.

```
/* Green for finished work, orange for work that is still open. */
.task-status.completed {
    background-color: #17a673;
}

.task-status.pending {
    background-color: #e08b1a;
}
```

These rules change **only** the background color.

- Completed badges become **green**.
- Pending badges become **orange**.

Everything else (white text, small size, pill shape) comes from `.task-status`.

This is why the two-class idea is useful. We write the shared styles one time, and only change what is different.

A small note: the grey color in `.task-status` is a backup color. In our project every badge also has `completed` or `pending`, so you never see the grey.

Remember the dot rule again:

```
.task-status.completed    /* both classes on one element - correct */
.task-status .completed   /* .completed inside .task-status - wrong here */
```

---

14. Flexbox Inside a Task Row

Each task row also uses Flexbox.

```
.task-item {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 15px;
    background-color: #ffffff;
    border: 1px solid #e2e5ea;
    border-radius: 10px;
    padding: 15px 18px;
    margin-bottom: 12px;
}
```

The result looks like this:

```
| Task title                Pending |
```

`display: flex;` Put the title and the badge in the same row.

`justify-content: space-between;` Put the title on one side and the badge on the other side. All the free space goes in the middle.

`align-items: center;` Keep them lined up in the middle from top to bottom. The badge is short and the title is taller, so without this they would not line up nicely.

Here is an easy way to remember:

- `justify-content` works **along** the row (left and right).
- `align-items` works **across** the row (up and down).

`gap: 15px;` Keep at least 15px between the title and the badge, so they never touch.

The other properties make the row look like a card:

- `background-color: #ffffff;` white background.
- `border: 1px solid #e2e5ea;` thin grey line around it.
- `border-radius: 10px;` round corners.
- `padding: 15px 18px;` space inside: 15px top and bottom, 18px left and right.
- `margin-bottom: 12px;` space **below** each row, so the rows do not touch each other.

Notice we used Flexbox **twice** in this project:

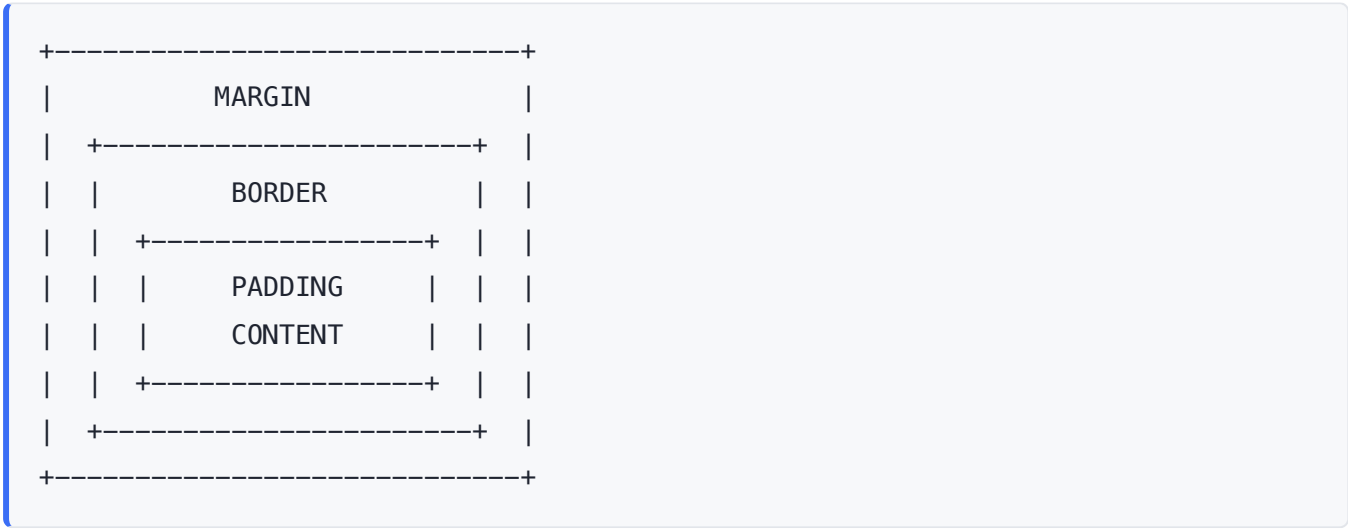
1. For the statistic cards (`.stats`).
2. For each task row (`.task-item`).

Same tool, two different jobs.

---

15. Margin vs Padding (The Box Model)

Every element is a box. Every box has four parts.



Content The text or the element itself.

Padding Space **inside** the element, between the content and the border.

Border The line around the element.

Margin Space **outside** the element, between this element and other elements.

An easy way to remember

Padding is inside. Margin is outside.

Padding pushes the border **away from** the text. Margin pushes other elements **away from** this box.

Padding is part of the box. If the box has a background color, the padding is colored too. Margin is empty space. It never has a color.

In TaskFlow

- `.stat-card` uses `padding: 20px;` This gives space **inside** the card, so the text does not touch the border.
- `.task-item` uses `margin-bottom: 12px;` This gives space **below** the row, so the rows do not touch each other.
- `.task-item` also uses `padding: 15px 18px;` This is the space **inside** the row.

If your text is touching the edge of a box, you need **padding**.

If two boxes are touching each other, you need **margin**.

---

16. CSS Shorthand

Shorthand means writing several values in one line.

Two values

```
padding: 30px 20px;
```

This means:

- `30px` = top and bottom
- `20px` = left and right

We use this in `.container`, `.header`, `.filter-button`, `.task-item` and `.task-status`.

One value

```
padding: 20px;
```

One value means **all four sides** are the same.

We use this in `.stat-card`.

margin: 0 auto

```
margin: 0 auto;
```

Same idea:

- `0` = top and bottom
- `auto` = left and right

`auto` is special. It shares the free space equally, so the box lands in the center.

border

```
border: 1px solid #e2e5ea;
```

This is also shorthand. It has three parts, always in this order:

- `1px` = how thick the line is
- `solid` = the style of the line (a normal line)
- `#e2e5ea` = the color

border-color

```
.filter-button:hover {
    border-color: #3b6ef5;
}
```

Here we change **only the color** of the border. The thickness and the style stay the same.

This is better than writing the whole `border` again.

---

17. Responsive Design

Responsive design means the page works on different screen sizes.

A phone screen is much narrower than a laptop screen. Three cards side by side would be far too narrow on a phone.

The media query

A **media query** lets us change the design when the screen becomes smaller.

Here is the real code from our project:

```
/* ---------- Small screens ---------- */

/* On a narrow screen there is no room for three cards in one row,
   so we turn the row into a column. */
@media (max-width: 600px) {

    .header h1 {
        font-size: 26px;
```

```
    }

    .stats {
        flex-direction: column;
    }

    .filters {
        flex-wrap: wrap;
    }

}
```

`@media (max-width: 600px)` These styles are used when the screen is **600px wide or smaller**.

On a bigger screen, the browser ignores everything inside.

What changes exactly

`.header h1 { font-size: 26px; }` The big title goes from 32px down to 26px, so it fits on a small screen.

`.stats { flex-direction: column; }` This is the main change.

`.stats` is still `display: flex`. We only change the **direction**.

- A row goes left to right.
- A column goes top to bottom.

So the three cards stack on top of each other:

```
Large screen:

[ Total ] [ Completed ] [ Pending ]


Small screen:

[ Total ]
[ Completed ]
[ Pending ]
```

The `gap: 16px` still works. Now it is the space between the cards from top to bottom.

`.filters { flex-wrap: wrap; }` This allows the buttons to move to a second line if there is not enough space. Without it, they would be squeezed together.

Two things to remember

1. Put media queries at the **end** of your CSS file. Rules that come later win, so the small-screen rules can change the normal rules.
2. We did not rewrite the layout. We already had Flexbox, so we only changed **one property** to get a completely different layout.

How to test it

Make your browser window narrow with the mouse.

Or press **F12** to open DevTools and click the small phone icon.

---

18. CSS Selectors Used in This Project

A **selector** is the part before the `{`. It says which elements to style.

Here are all the kinds we used:

`*` Select all elements.

`body` Select the `<body>` element. This is a **tag** selector. It has no dot.

`.container` Select elements with `class="container"`. This is a **class** selector. It has a dot.

`.header h1` A space means "inside". Select the `<h1>` that is inside `.header`.

`.filter-button.active` No space means "both". Select an element that has `class="filter-button active"`.

`.filter-button:hover` Select a filter button while the mouse is over it.

`.task-status.completed` Select an element that has both `task-status` and `completed`.

`@media (max-width: 600px)` This is not a selector for elements. It is a rule that says "use these styles only on small screens".

The most important rule to remember

> A SPACE means INSIDE.
> NO SPACE means BOTH ON THE SAME ELEMENT.

Getting this wrong is the most common CSS mistake for beginners.

---

19. CSS Properties We Used

Here is every important property from our Session 1 CSS.

Property	What it means	Where we use it
`margin`	Space outside an element	`*`, `.container`
`margin-top`	Space above an element	`.header p`, `.stat-value`, `.filters`, `.task-list`
`margin-bottom`	Space below an element	`.task-item`
`padding`	Space inside an element	`*`, `.header`, `.container`, `.stat-card`, `.filter-button`, `.task-item`, `.task-status`
`box-sizing`	Keeps padding inside the width	`*`
`font-family`	Which font to use	`body`
`font-size`	How big the text is	`.header h1`, `.stat-label`, `.stat-value`, `.filter-button`, `.task-status`
`font-weight`	How thick the text is	`.stat-value`
`line-height`	Space between lines of text	`body`
`color`	The text color	`body`, `.header h1`, `.header p`, `.stat-label`, `.filter-button`, `.task-status`
`background-color`	The background color	`body`, `.header`, `.stat-card`, `.filter-button`, `.task-item`, `.task-status`
`border`	A line around an element	`.stat-card`, `.filter-button`, `.task-item`
`border-bottom`	A line only under an element	`.header`
`border-color`	Changes only the border color	`.filter-button:hover`

Property	What it means	Where we use it
`border-radius`	Round corners	`.stat-card`, `.filter-button`, `.task-item`, `.task-status`
`text-align`	Puts text left, right or center	`.header`, `.stat-card`
`max-width`	Never wider than this	`.container`
`display: flex`	Put the children in a row	`.stats`, `.filters`, `.task-item`
`gap`	Space between flex children	`.stats`, `.filters`, `.task-item`
`flex: 1`	Take an equal share of the row	`.stat-card`
`flex-direction`	Row or column	`.stats` (small screens)
`flex-wrap`	Allow a second line	`.filters` (small screens)
`justify-content`	Position along the row	`.task-item`
`align-items`	Position across the row	`.task-item`
`list-style`	Shows or hides list bullets	`.task-list`
`cursor`	The mouse pointer shape	`.filter-button`

---

20. Complete HTML Walkthrough

Now let us read the whole `index.html` from top to bottom.

Part 1: The start of the page

```
<!DOCTYPE html>
<html lang="en">

<head>
```

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>TaskFlow</title>

    <link rel="stylesheet" href="css/style.css">
</head>
```

What it does: starts the page and loads the CSS file.

What you see: nothing yet. The `<head>` is never shown on the screen. But the browser tab now says "TaskFlow".

Part 2: The header

```
<body>

    <!-- The top bar of the application. -->
    <header class="header">
        <h1>TaskFlow</h1>
        <p>All your tasks in one place.</p>
    </header>
```

What it does: creates the top bar with the title and one line of text.

What you see: a white bar across the top. Blue "TaskFlow" in the middle, grey text under it, and a thin line at the bottom of the bar.

Part 3: The main container

```
    <main class="container">
```

What it does: opens the box that holds all the main content.

What you see: nothing by itself. It is an invisible box. But it keeps everything inside it in the center of the screen.

Part 4: The statistic cards

```
        <!-- The cards that show our numbers. -->
        <section class="stats">

            <div class="stat-card">
                <p class="stat-label">Total Tasks</p>
                <p class="stat-value">6</p>
            </div>

            <div class="stat-card">
                <p class="stat-label">Completed</p>
                <p class="stat-value">2</p>
            </div>

            <div class="stat-card">
                <p class="stat-label">Pending</p>
                <p class="stat-value">4</p>
            </div>

        </section>
```

What it does: makes three cards with a small label and a big number.

What you see: three white cards in one row. They are all the same width. Each has small grey text on top and a big black number under it.

Remember: these numbers are typed by hand.

Part 5: The filter buttons

```
        <!-- The buttons that will choose which tasks we see. -->
        <section class="filters">
            <button class="filter-button active">All</button>
            <button class="filter-button">Completed</button>
            <button class="filter-button">Pending</button>
        </section>
```

What it does: makes three buttons in a row.

What you see: three buttons under the cards. "All" is blue with white text. The other two are white with grey borders. When you move the mouse over a button, its border turns blue.

Clicking does nothing yet.

Part 6: The task list

```
        <!-- The list of tasks. -->
        <ul class="task-list">

            <li class="task-item">
                <span class="task-title">Design the TaskFlow page</span>
                <span class="task-status completed">Completed</span>
            </li>
```

(The file has six `<li>` blocks like this one.)

What it does: makes one row for each task.

What you see: white rows with round corners. The task name is on the left. A small colored badge is on the right. Green badges say "Completed". Orange badges say "Pending".

Part 7: The end of the page

```
        </ul>

    </main>

    <script src="js/app.js"></script>

</body>

</html>
```

What it does: closes the list, closes the main box, loads the JavaScript file, and ends the page.

What you see: no change. The JavaScript file is empty in Session 1.

We put `<script>` at the bottom so the browser reads all the HTML first.

---

21. Complete CSS Walkthrough

Now the whole `style.css`, section by section.

Section 1: Basic setup

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

Selects: every element on the page.

Does: removes the browser's default spacing, and keeps padding and border inside the width.

You see: the content moves to the very top-left corner. We start from a clean page.

```
body {
    font-family: Arial, Helvetica, sans-serif;
    background-color: #f4f5f7;
    color: #1f2430;
    line-height: 1.5;
}
```

Selects: the whole visible page.

Does: sets the font, the light grey background, the dark text color and the line spacing.

You see: the page background turns light grey and all text uses the same font.

Section 2: Header

```
.header {
    background-color: #ffffff;
    border-bottom: 1px solid #e2e5ea;
    padding: 30px 20px;
    text-align: center;
}
```

Selects: the element with `class="header"`.

Does: makes it a white bar with space inside, centered text, and a line under it.

You see: a clear white bar at the top of the page.

```
.header h1 {
    color: #3b6ef5;
    font-size: 32px;
}
```

Selects: the `<h1>` inside `.header`.

Does: makes the title blue and big.

You see: "TaskFlow" is now large and blue.

```
.header p {
    color: #6b7280;
    margin-top: 6px;
}
```

Selects: the `<p>` inside `.header`.

Does: makes the text grey and adds a small space above it.

You see: the small line under the title looks softer and less important.

Section 3: Page container

```
.container {
    max-width: 700px;
    margin: 0 auto;
    padding: 30px 20px;
}
```

Selects: the `<main>` element.

Does: limits the width, centers the box, and adds space inside.

You see: on a wide screen the content no longer stretches to the edges. It sits in a neat column in the middle.

Section 4: Statistic cards

```
.stats {
    display: flex;
    gap: 16px;
}
```

Selects: the box that holds the three cards.

Does: puts the cards in one row with space between them.

You see: the cards move from a vertical stack into a horizontal row.

```
.stat-card {
    flex: 1;
    background-color: #ffffff;
    border: 1px solid #e2e5ea;
    border-radius: 10px;
    padding: 20px;
    text-align: center;
}
```

Selects: each of the three cards.

Does: gives them equal width, a white background, a thin border, round corners, space inside, and centered text.

You see: three equal white cards with round corners.

```
.stat-label {
    color: #6b7280;
    font-size: 14px;
}

.stat-value {
    font-size: 30px;
    font-weight: bold;
    margin-top: 5px;
}
```

Selects: the small label and the big number inside each card.

Does: makes the label small and grey, and the number big and bold.

You see: your eye goes to the number first. That is what we want.

Section 5: Filter buttons

```
.filters {
    display: flex;
    gap: 10px;
    margin-top: 30px;
}
```

Selects: the box that holds the buttons.

Does: puts them in a row with space between, and adds space above the group.

You see: three buttons in a line, with a clear gap above them.

```
.filter-button {
    background-color: #ffffff;
    border: 1px solid #e2e5ea;
    border-radius: 8px;
    color: #1f2430;
    cursor: pointer;
    font-size: 15px;
    padding: 10px 18px;
}
```

Selects: all three buttons.

Does: gives them a white background, a thin border, slightly round corners, a hand cursor and space inside.

You see: the buttons look like real buttons, and the mouse becomes a hand over them.

```
.filter-button:hover {
    border-color: #3b6ef5;
}
```

Selects: a button while the mouse is over it.

Does: changes the border color to blue.

You see: the border turns blue when you point at a button, then goes back.

```
.filter-button.active {
    background-color: #3b6ef5;
    border-color: #3b6ef5;
    color: #ffffff;
}
```

Selects: a button that has both `filter-button` and `active`.

Does: makes it blue with white text.

You see: only the "All" button is blue, because only it has the `active` class.

Section 6: Task list

```
.task-list {
    list-style: none;
    margin-top: 20px;
}
```

Selects: the `<ul>`.

Does: removes the bullet points and adds space above the list.

You see: the small dots in front of each task disappear.

```
.task-item {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 15px;
    background-color: #ffffff;
    border: 1px solid #e2e5ea;
    border-radius: 10px;
    padding: 15px 18px;
    margin-bottom: 12px;
}
```

Selects: each `<li>`.

Does: puts the title and the badge in one row, pushes them to opposite sides, lines them up in the middle, and makes the row look like a card.

You see: neat white rows. Title on the left, badge on the right, space between rows.

```
.task-status {
    border-radius: 20px;
    color: #ffffff;
    background-color: #6b7280;
    font-size: 12px;
    padding: 5px 12px;
}
```

Selects: every badge.

Does: makes a small pill shape with white text.

You see: a small rounded label at the end of each row.

```
.task-status.completed {
    background-color: #17a673;
}

.task-status.pending {
    background-color: #e08b1a;
}
```

Selects: badges that also have `completed` or `pending`.

Does: changes only the background color.

You see: green badges for finished tasks, orange badges for open tasks.

Section 7: Small screens

```
@media (max-width: 600px) {

    .header h1 {
        font-size: 26px;
    }

    .stats {
        flex-direction: column;
    }

    .filters {
        flex-wrap: wrap;
```

```
    }

}
```

Selects: nothing by itself. It turns on these rules only when the screen is 600px or smaller.

Does: makes the title smaller, turns the card row into a column, and lets the buttons move to a second line.

You see: on a narrow window the three cards stack on top of each other.

---

22. app.js at the End of Session 1

Here is the complete file:

```
/* TaskFlow JavaScript.
   This file is empty for now. We start using it in Session 2. */
```

That is everything. The file has **only a comment** inside.

A comment is a note for people. The browser ignores it.

So this file does **nothing** right now.

We already created `app.js` because TaskFlow will use JavaScript later.

We also already connected it in the HTML:

```
<script src="js/app.js"></script>
```

This means everything is ready. In Session 2 we only have to write code, not set up files.

In Session 1, JavaScript is not controlling the page.

We will start using it in Session 2.

---

23. What Happens When We Open TaskFlow?

Here is what the browser does, step by step:

```
Browser opens index.html
          |
          v
```

```
     Reads the HTML
           |
           v
  Loads css/style.css
           |
           v
Matches CSS with the HTML
           |
           v
   Shows the styled page
```

Let us say it in words.

1. You open `index.html`.
2. The browser reads the HTML and learns what is on the page.
3. It sees `<link rel="stylesheet" href="css/style.css">` and loads the CSS file.
4. For every CSS rule, it looks for elements that match the selector. For example, it finds all elements with `class="stat-card"`.
5. It applies the styles and shows the finished page.

If step 3 fails, because the path is wrong, you still see all the text, but with no colors, no cards and no layout.

That plain white page is a very useful clue. It usually means the CSS file was not found.

---

24. Common Beginner Mistakes

These are real mistakes that happen to everyone. Here is how to fix them.

1. The class name does not match

HTML:

```
<div class="stat-cards">
```

CSS:

```
.stat-card {
    padding: 20px;
}
```

Problem: the HTML says `stat-cards` and the CSS says `stat-card`. The names are different, so the CSS will not work.

Fix: make both names exactly the same.

CSS never shows an error for this. It just does nothing.

2. Forgetting the dot in CSS

```
stat-card {        /* wrong */
    padding: 20px;
}

.stat-card {       /* correct */
    padding: 20px;
}
```

Problem: without the dot, CSS looks for a `<stat-card>` tag, which does not exist.

Fix: put a dot before class names in CSS.

Remember: dot in the CSS file, no dot in the HTML file.

3. A space in the selector by mistake

```
.task-status .completed {   /* wrong for our HTML */
    background-color: #17a673;
}

.task-status.completed {    /* correct */
    background-color: #17a673;
}
```

Problem: a space means "inside". Our two classes are on the same element.

Fix: remove the space.

This is the most common CSS mistake in this project.

4. Wrong CSS file path

```
<link rel="stylesheet" href="style.css">        <!-- wrong -->
<link rel="stylesheet" href="css/style.css">    <!-- correct -->
```

Problem: our file is inside the `css` folder, so the folder name must be in the path.

Fix: write the folder name too.

How to spot it: the page has no styles at all.

5. Missing closing tag

```
<div class="stat-card">
    <p class="stat-label">Total Tasks</p>
    <p class="stat-value">6</p>
<!-- the </div> is missing -->
```

Problem: the browser gets confused about where the box ends. The layout can break in strange ways.

Fix: every opening tag needs a closing tag. Use the indentation to check. Things that line up should match.

6. Missing a closing curly bracket

```
.stat-card {
    padding: 20px;
/* the } is missing */

.stat-label {
    color: #6b7280;
}
```

Problem: everything after the missing `}` stops working.

Fix: every `{` needs a `}`. In VS Code, click on a bracket and it shows you its partner.

7. Missing a semicolon

```
.stat-card {
    padding: 20px      /* the ; is missing */
    text-align: center;
}
```

Problem: the browser cannot read this line and the next one.

Fix: end every line inside `{ }` with `;`.

8. CSS written outside the curly brackets

```
.stat-card {
    padding: 20px;
}
    text-align: center;     /* wrong - this is outside */
```

Problem: CSS properties must be inside a rule.

Fix: move the line inside the `{ }`.

9. Confusing margin and padding

Problem: the text touches the border of the card, and you add `margin`. Nothing changes inside the card.

Fix: space **inside** needs `padding`. Space **outside** needs `margin`.

10. display: flex on the wrong element

```
.stat-card {
    display: flex;      /* wrong element */
}

.stats {
    display: flex;      /* correct - the parent */
}
```

Problem: Flexbox arranges the **children** of the element you put it on. Putting it on the card arranges the things inside the card, not the cards themselves.

Fix: put `display: flex` on the **parent**, the box that contains the items.

11. Forgetting to save

Problem: you changed the file but the browser shows the old page.

Fix: press `Cmd + S` or `Ctrl + S`. In VS Code, a white dot next to the file name means it is not saved.

12. Forgetting to refresh

Problem: you saved, but the browser still shows the old page.

Fix: refresh with `Cmd + R` or `F5`.

If you use Live Server, it refreshes by itself.

---

25. Simple Debugging Checklist

When something does not work, go through this list in order.

1. Save the file. An unsaved change does nothing. Look for the white dot in VS Code.

2. Refresh the browser. The browser may still show the old version.

3. Check the class name. Read the name in the HTML and in the CSS. They must match exactly.

4. Check spelling. `center` not `centre`. `color` not `colour`. CSS uses American spelling.

5. Check the CSS selector. Is the dot there? Is there a space that should not be there?

6. Check `{` and `}`. Every open bracket needs a closing bracket. Look above the broken part.

7. Check the CSS path. If the whole page has no styles, the path in `<link>` is probably wrong.

8. Open DevTools. Press `F12`, or right-click the page and choose Inspect.

9. Inspect the element. Right-click the broken part and choose Inspect. Look at the Styles panel on the right.

- Is your rule there? Good, the selector works.
- Is it crossed out? Another rule won.
- Is it missing completely? Your selector matched nothing. Go back to step 3.

10. Check for errors. Look at the Console tab in DevTools.

Remember: **CSS never tells you when a name is wrong. It just does nothing.** That is why we inspect.

26. Quick Review Questions

Try to answer these without looking. The answers are below.

1. What does HTML do?
2. What does CSS do?
3. What is a class?
4. Why does `.stat-card` start with a dot?
5. What is the difference between margin and padding?
6. What does `display: flex` do, and which element do we put it on?
7. What does `gap` do?
8. What does `flex: 1` do?
9. Why do we use `max-width` instead of `width` on the container?
10. What does `margin: 0 auto` do?
11. What does `:hover` mean?
12. What is a media query?
13. What happens to the cards when the screen becomes smaller than 600px?
14. What is the difference between `.task-status.completed` and `.task-status .completed`?
15. Do the filter buttons work at the end of Session 1?

Answers

1. HTML says what is on the page. It is the structure and the content.

2. CSS says how the page looks. Colors, sizes, spacing and layout.

3. A class is a name we give to an element, so CSS can find it and style it. Many elements can share the same class.

4. In CSS, a dot means "class". Without the dot, CSS looks for a tag with that name.

5. Padding is space **inside** the element. Margin is space **outside** it.

6. It puts the children in a row. We put it on the **parent**, the box that contains them.

7. It adds space between the flex children. In `.stats` it is 16px between the cards.

8. It gives each card an equal share of the row, so all three are the same width.

9. `max-width` means "never wider than this, but smaller is fine". `width` would force the size and break the page on a phone.

10. `0` is the top and bottom space. `auto` shares the free space on the left and right equally, so the box sits in the center.

11. It means "while the mouse is over this element".

12. A media query lets us use different styles at different screen sizes. Ours is `@media (max-width: 600px)`.

13. `.stats` changes from a row to a column, so the three cards stack on top of each other. The title also gets smaller.

14. No space means both classes are on the **same** element. That is what we want. A space means the second class is **inside** the first element. That is different.

15. No. They look like buttons and they react to the mouse, but clicking them does not change the list. We will make them work in Session 2.

---

27. Beginner Glossary

HTML The language that says what is on the page.

CSS The language that says how the page looks.

element One part of the page, like a heading, a button or a box.

tag The code that makes an element, like `<p>` or `<button>`. Most tags come in pairs: an opening tag and a closing tag.

attribute Extra information inside a tag, like `class="header"` or `href="css/style.css"`.

class A name we give to an element so CSS can find it. Many elements can share one class.

selector The part of a CSS rule that says which elements to style, like `.stat-card`.

property The thing you want to change, like `color` or `padding`.

value What you set the property to, like `20px` or `#ffffff`.

rule A selector plus its properties inside `{ }`.

container A box that holds other elements. In our project `.container` holds the main content.

parent An element that contains other elements.

child An element that is inside another element.

Flexbox A CSS tool that puts elements next to each other in a row or a column. You turn it on with `display: flex` on the parent.

responsive design Making the page work well on different screen sizes.

media query A CSS rule that uses different styles at different screen sizes, like `@media (max-width: 600px)`.

hover When the mouse is over an element.

box model The idea that every element is a box with content, padding, border and margin.

margin Space outside an element.

padding Space inside an element.

border The line around an element.

hex color A color written as a code with a `#`, like `#3b6ef5`.

comment A note in the code for people. The browser ignores it. In HTML: `<!-- like this -->`. In CSS: `/* like this */`.

DevTools The tools inside the browser that let you look at your page. Open them with `F12`.

You Are Done

You now know every line of the Session 1 TaskFlow project.

You can:

- build a page structure with HTML
- style it with CSS classes
- put things in a row with Flexbox
- control space with margin and padding
- make the page work on a small screen
- find and fix simple mistakes

In Session 2, JavaScript will start to calculate the numbers and build the task list for us.

See you next session.